TASK 11:

For the above application create authorized end points using JWT (JSON Web Token).

Steps :

1. create a project and install the libraries (be in the project path)

```
npm install mysql2
npm install jsonwebtoken
```

2. Create a db collegedb5_ _ and table studentdetails1(id, name , course) same as task 10

3. store app.js file in the project folder

app.js

```
const express = require('express');
const mysql = require('mysql2');
const jwt = require('jsonwebtoken'); // 1. Import JWT
const app = express();
app.use(express.json());
const SECRET_KEY = "mgit_secret_key";
const pool = mysql.createPool({
  host: '127.0.0.1',
  user: 'root',
  password: '  ',  //msql pwd
  database: 'collegedb1',
  port: 3307
}).promise();
```

```
const authenticateToken = (req, res, next) => {
   const authHeader = req.headers['authorization'];
   const token = authHeader && authHeader.split(' ')[1]; // Format: "Bearer TOKEN"
   if (!token) return res.status(401).json({ message: "Access Denied: No Token Provided" });
   jwt.verify(token, SECRET_KEY, (err, user) => {
      if (err) return res.status(403).json({ message: "Invalid or Expired Token" });
      req.user = user;
      next(); // Move to the actual route logic
   });
};

// LOGIN ROUTE: Generates a token (Simple version for your lab)
app.post('/login', (req, res) => {
   const { username, password } = req.body;
   if (username === 'admin' && password === 'password123') {
      const token = jwt.sign({ username }, SECRET_KEY, { expiresIn: '7d' });
      return res.json({ token });
   }
   res.status(401).json({ message: "Invalid Credentials" });
});

// PROTECTED ROUTES: Added 'authenticateToken' as a second parameter
app.get('/studentdetails1', authenticateToken, async (req, res) => {
   try {
      const [rows] = await pool.query('SELECT * FROM studentdetails1');
      res.json(rows);
   } catch (err) {
      res.status(500).json({ message: err.message });
```

```
    }
});

app.post('/studentdetails1', authenticateToken, async (req, res) => {
    const { name, course } = req.body;
    try {
        const [result] = await pool.execute(
            'INSERT INTO studentdetails1 (name, course) VALUES (?, ?)',
            [name, course]
        );
        res.status(201).json({ id: result.insertId, name, course });
    } catch (err) {
        res.status(500).json({ error: err.message });
    }
});
app.put('/studentdetails1/:id', authenticateToken, async (req, res) => {
    const { id } = req.params;
    const { name, course } = req.body;
    try {
        const [result] = await pool.execute(
            'UPDATE studentdetails1 SET name = ?, course = ? WHERE id = ?',
            [name, course, id]
        );
        if (result.affectedRows === 0) return res.status(404).json({ message: "Not found" });
        res.json({ message: "Updated successfully", id });
    } catch (err) {
        res.status(500).json({ error: err.message });
    }
});

app.delete('/studentdetails1/:id', authenticateToken, async (req, res) => {
```

```
  const { id } = req.params;
  try {
    const [result] = await pool.execute('DELETE FROM studentdetails1 WHERE id = ?', [id]);
    if (result.affectedRows === 0) return res.status(404).json({ message: "Not found" });
    res.json({ message: `Student ${id} deleted` });
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});
app.listen(3000, () => console.log('Server running on port 3000'));
```

4. compile the program

   node app.js

5. Open postman and enter 127.0.0.1/login

Select body->raw->Jason

And provide the credentials (username & password given in the program) to generate the token.

6. Copy the token and use it for all the methods

By selecting authorization->type->bearer token.